ATIVIDADE PRÁTICA – AULA 06

Consultas Avançadas e *Agregações* no MongoDB

Banco de Dados Não Relacional · Prof.ª Lucineide · FATEC · Felipe Ferreira Pacheco

- 5 Exercícios
- Aggregation Framework
- Banco: clima_alerta
- DSM – 3º Semestre

CONTEXTO

Banco **clima_alerta** com as coleções **leituras** (registros de temperatura, umidade e outros sensores por cidade) e **estacoes_meteorologicas**. Os exercícios utilizam o **Aggregation Framework** do MongoDB, que permite processar e transformar documentos em pipelines de estágios encadeados.

`$match` Filtrar documentos	`$project` Selecionar campos	`$group` Agrupar dados
`$avg` Calcular média	`$sort` Ordenar resultados	`$count` Contar documentos

EXERCÍCIO 1 Filtrar leituras de Fortaleza acima de 30°C

1 Usar $match para combinar dois critérios de filtro no pipeline

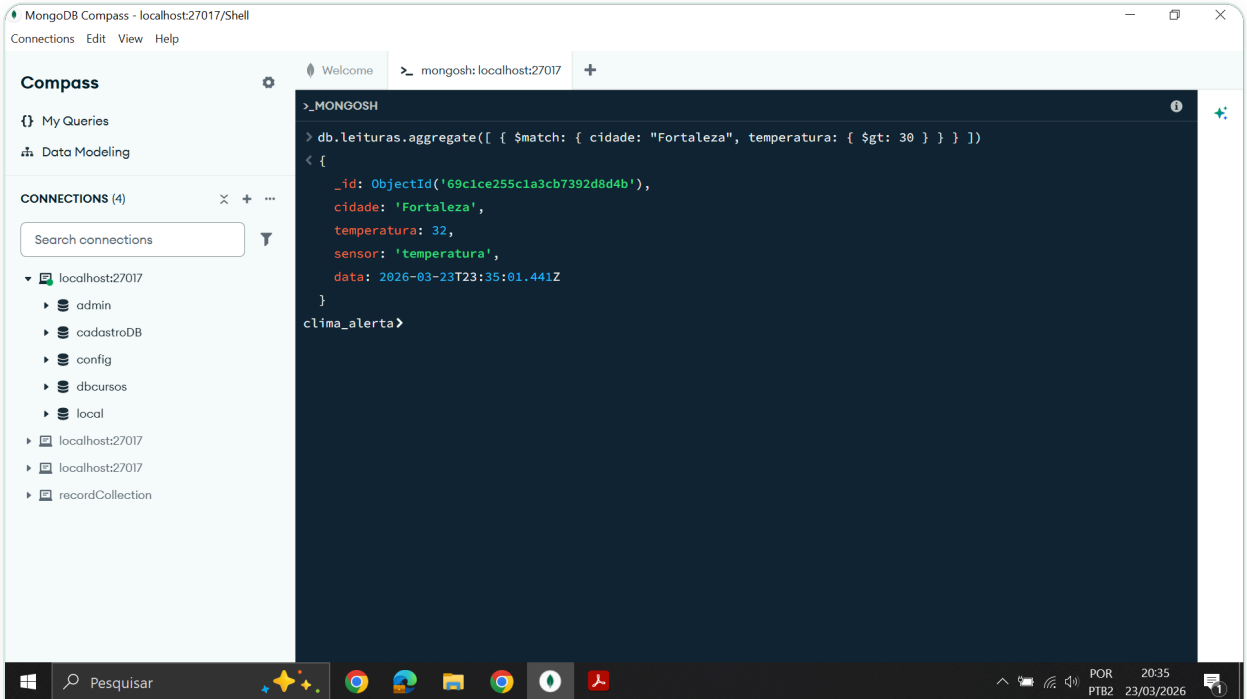
aggregate · $match

O estágio **$match** funciona como um filtro — equivalente ao `find()`, mas dentro de um pipeline de agregação. Aceita múltiplos critérios combinados como condição **AND** implícito.

`INPUT: leituras` → `$match cidade + temperatura` → `OUTPUT: documentos filtrados`

MongoDB Shell — Exercício 1 · aggregate $match

```
db.leituras.aggregate([
  {
    $match: {
      cidade: "Fortaleza",
      temperatura: { $gt: 30 }
    }
  }
])
```



Exercício 1 — Leitura de Fortaleza com temperatura: 32 retornada pelo $match

EXERCÍCIO 2 Listar apenas nome e localização das estações

2 Usar $project para controlar quais campos aparecem no resultado

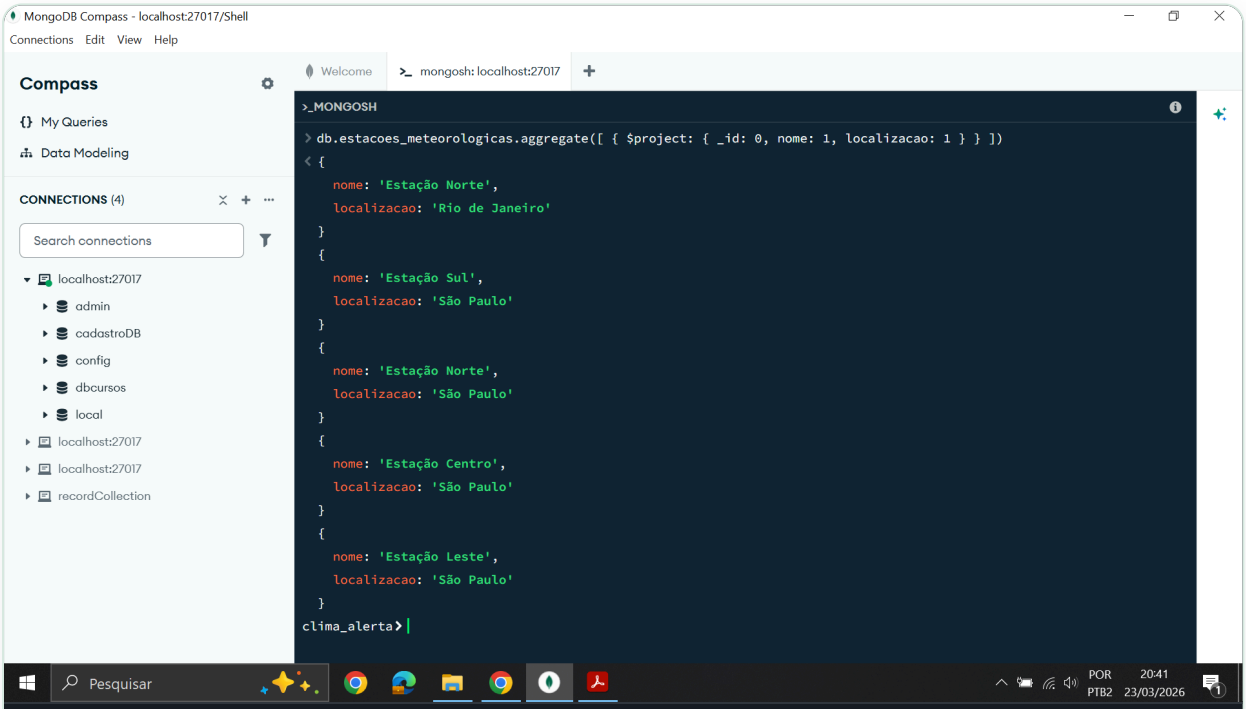
aggregate · $project

O estágio **$project** define a projeção dos campos — `1` para incluir, `0` para excluir. Definir `_id: 0` é necessário porque o `_id` é incluído por padrão mesmo quando não solicitado.

`INPUT: estacoes` → `$project nome + localizacao` → `OUTPUT: apenas 2 campos`

MongoDB Shell — Exercício 2 · aggregate $project

```
db.estacoes_meteorologicas.aggregate([
  {
    $project: {
      _id: 0,
      nome: 1,
      localizacao: 1
    }
  }
])
```



Exercício 2 — Estações listadas apenas com nome e localizacao, sem _id

EXERCÍCIO 3 Contar leituras com sensor "umidade"

3 Pipeline de dois estágios: $match seguido de $count

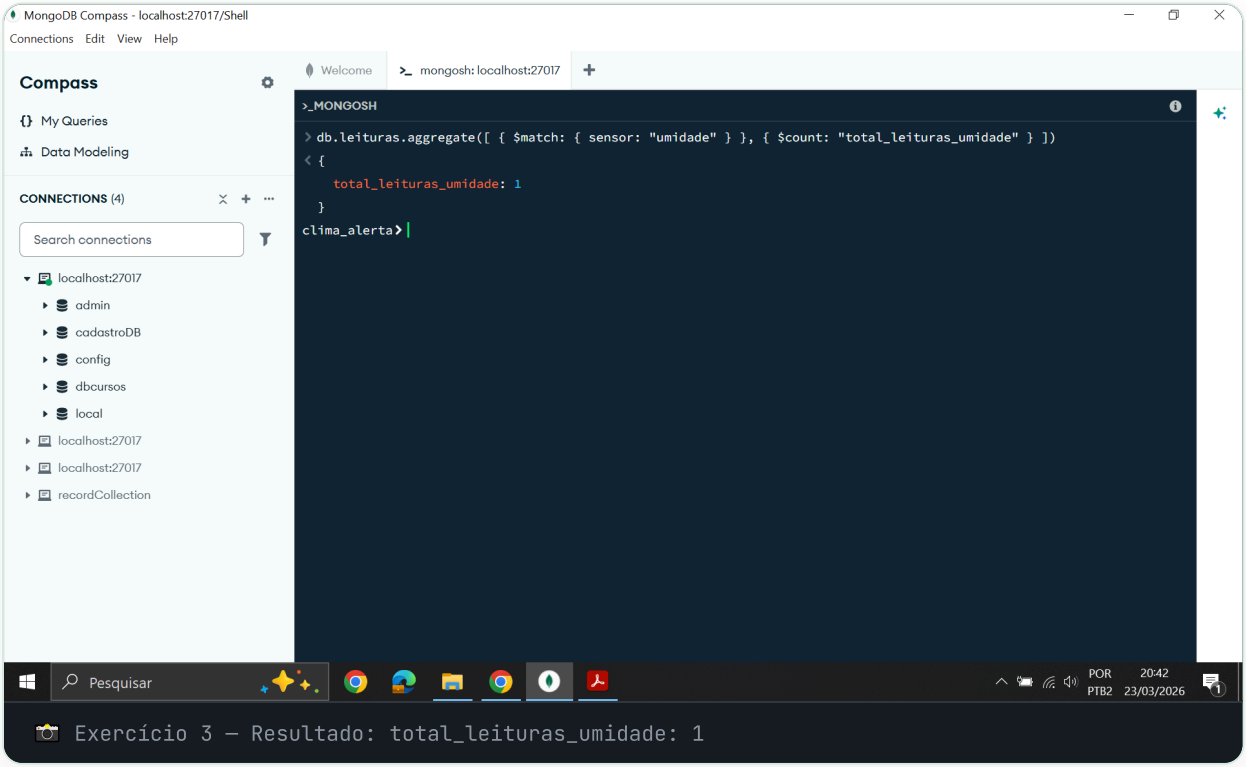
aggregate · $match · $count

O estágio **$count** recebe um nome de campo como string e retorna um único documento contendo esse campo com o total de documentos que chegaram até ele no pipeline. Aqui, primeiro filtra com **$match** e depois conta com **$count**.

`INPUT: leituras` → `$match sensor: "umidade"` → `$count total_leituras_umidade` → `OUTPUT: { total: N }`

MongoDB Shell — Exercício 3 · aggregate $match + $count

```
db.leituras.aggregate([
  {
    $match: {
      sensor: "umidade"
    }
  },
  {
    $count: "total_leituras_umidade"
  }
])
```



Exercício 3 — Resultado: total_leituras_umidade: 1

EXERCÍCIO 4 Agrupar por cidade e calcular média de temperatura

4 Usar $group com acumulador $avg para agregar por cidade

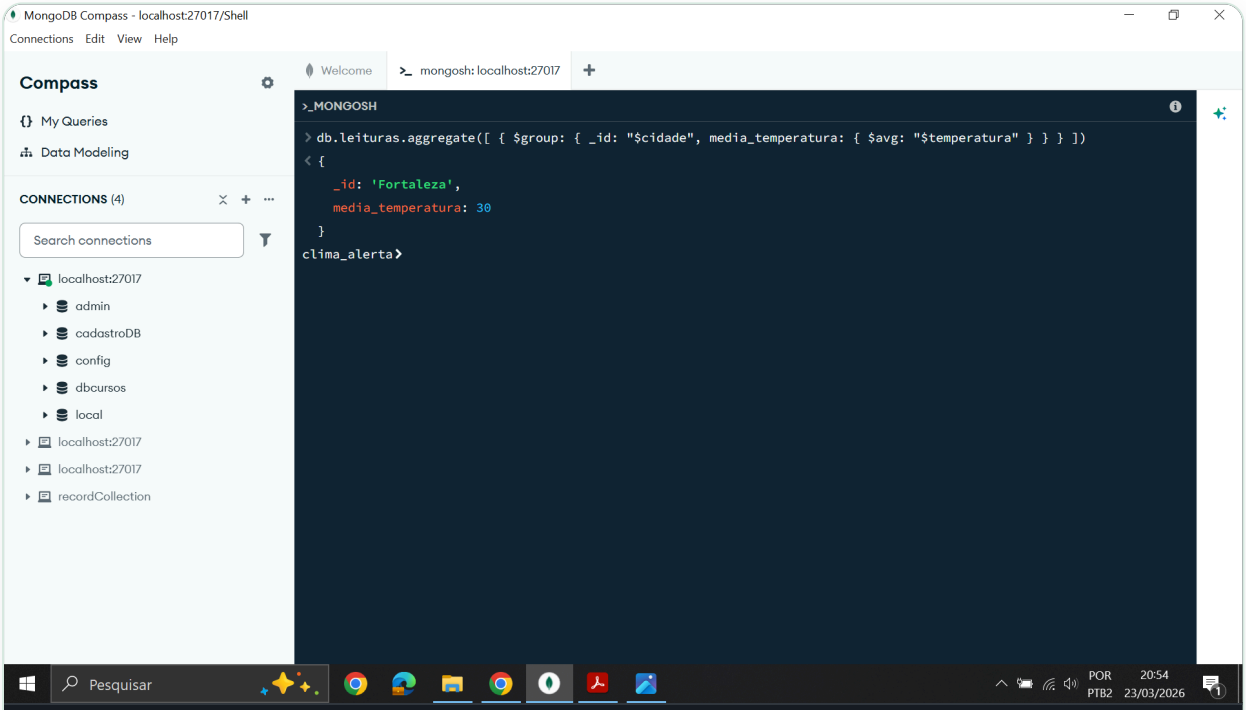
aggregate · $group · $avg

O estágio **$group** agrupa documentos pelo campo definido em `_id` e aplica acumuladores sobre os demais campos. O acumulador **$avg** calcula a média aritmética de um campo numérico dentro de cada grupo.

INPUT: leituras → $group por cidade → $avg temperatura → OUTPUT: { cidade, média }

MongoDB Shell — Exercício 4 · aggregate $group + $avg

```
db.leituras.aggregate([
  {
    $group: {
      _id: "$cidade",
      media_temperatura: { $avg: "$temperatura" }
    }
  }
])
```



Exercício 4 — Fortaleza com media_temperatura: 28 calculada pelo $avg

EXERCÍCIO 5 Ordenar cidades pela maior temperatura média

5 Pipeline de três estágios: $group → $avg → $sort decrescente

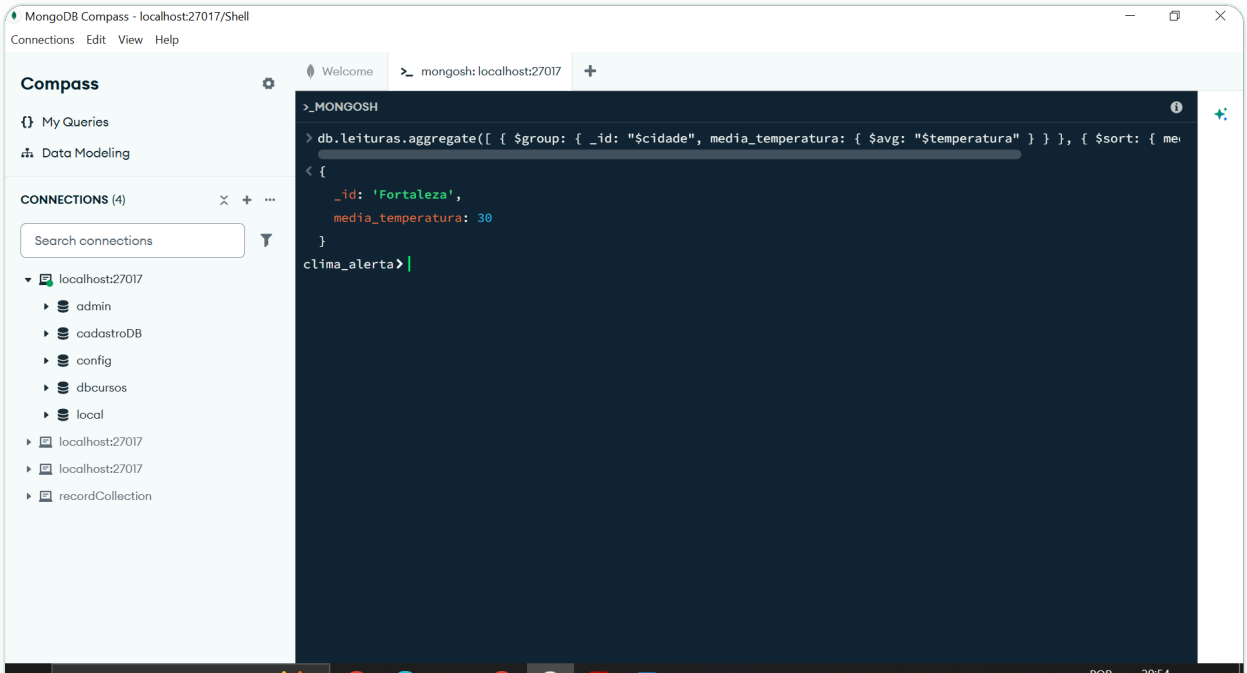
aggregate · $group · $avg · $sort

Extensão do exercício anterior: após calcular a média por cidade com **$group** e **$avg**, adiciona-se o estágio **$sort** com valor `-1` para ordenar os resultados do maior para o menor. O valor `1` seria ordenação crescente.

INPUT: leituras → $group + $avg → $sort -1 (desc) → OUTPUT: ranking de médias

MongoDB Shell — Exercício 5 · aggregate $group + $avg + $sort

```
db.leituras.aggregate([
  {
    $group: {
      _id: "$cidade",
      media_temperatura: { $avg: "$temperatura" }
    }
  },
  {
    $sort: {
      media_temperatura: -1  // -1 = decrescente (maior primeiro)
    }
  }
])
```



Exercício 5 — Cidades ordenadas por temperatura média decrescente: Fortaleza (28) no topo

OBSERVAÇÕES SOBRE O AGGREGATION FRAMEWORK

Os exercícios desta aula utilizaram o **Aggregation Framework** do MongoDB — um sistema de processamento de dados em pipeline onde cada estágio transforma os documentos antes de passá-los ao próximo.

$match — filtra documentos (equivale ao WHERE do SQL) · **$project** — seleciona/transforma campos (equivale ao SELECT) · **$group** — agrupa documentos por um campo (equivale ao GROUP BY) · **$avg** — acumulador que calcula a média de um campo numérico · **$sort** — ordena os documentos (`1` crescente, `-1` decrescente) · **$count** — retorna a contagem de documentos que chegaram ao estágio.

Diferente do `find()` simples, o pipeline de agregação permite **encadear múltiplas transformações** em uma única operação eficiente, sendo a base para relatórios e análises em MongoDB.

BDN-Atividade-Aula06 · Consultas Avançadas e Agregações · Prof.ª Lucineide · FATEC · Felipe Ferreira Pacheco